# 講義 3：Local Type（程式裡自己定義的型別）與 Structure（結構）

> 對應練習：ex03 | 答案程式：ZR_TR03_STRUCTURES

## 本講重點

- 為什麼需要結構（structure）：一筆資料有多個欄位
- `TYPES BEGIN OF ... END OF` 定義自訂結構型別
- 「型別（TYPES）」與「變數（DATA）」的分工：模具 vs 產品
- 欄位存取 `-`、結構整體賦值、`CLEAR`
- `MOVE-CORRESPONDING` 同名欄位搬值

## 1. 為什麼需要結構

一個學生有學號、姓名、成績三個欄位。用三個獨立變數也能寫，但「這三個變數屬於同一筆資料」的關係只存在你腦中，程式看不出來，傳遞時也得三個三個搬。結構把相關欄位**打包成一筆**：

```
* 沒有結構：三個散裝變數
DATA: gv_id    TYPE c LENGTH 5,
      gv_name  TYPE string,
      gv_score TYPE i.

* 有結構：一筆資料一個變數
DATA gs_student TYPE ty_student.
```

結構是下一講 internal table（多筆資料）的基礎：**結構是一列，internal table 是很多列**。

## 2. TYPES：定義自訂結構型別

```
TYPES: BEGIN OF ty_student,
         id    TYPE c LENGTH 5,     " 學號
         name  TYPE string,         " 姓名
         score TYPE i,              " 成績
       END OF ty_student.
```

語法要點：

- `BEGIN OF 名稱` 開始、`END OF 名稱` 結束，中間每行是一個欄位（component），格式同 DATA 的型別描述。
- 慣例：結構型別取名 `ty_` 開頭；之後的表格型別取 `tt_` 開頭。
- 定義在程式開頭（宣告區），這種只在本程式看得到的型別叫 **Local Type**。多支程式要共用時，型別改定義在資料字典（SE11 的 Structure / Table Type），那叫 **Global Type**（講義 6 第 1.1 節詳解）；本講先用 Local Type 學結構語法。另外，結構裡代表業務資料的欄位，實務上同樣是引用資料字典的欄位型別（如 `carrid TYPE scarr-carrid`），本講範例為了不依賴資料表才寫成內建型別。

## 3. TYPES vs DATA：模具與產品

| | TYPES | DATA |
|---|---|---|
| 產生什麼 | 型別（描述長相，**不佔記憶體**） | 資料物件（真的可以存值） |
| 比喻 | 餅乾模具 | 壓出來的餅乾 |
| 可以賦值嗎 | 不可以 | 可以 |

```
TYPES ty_amount TYPE p LENGTH 8 DECIMALS 2.    " 模具：定義一次
DATA: gv_price TYPE ty_amount,                 " 產品：想壓幾個壓幾個
      gv_cost  TYPE ty_amount,
      gv_total TYPE ty_amount.
```

> 初學常犯：對 TYPES 定義的名字賦值（`ty_student-id = ...`）——編譯錯誤，型別不能存資料，要先 `DATA gs_student TYPE ty_student.` 造出變數。

## 4. 欄位存取：連字號 `-`

```
DATA gs_student TYPE ty_student.

gs_student-id    = 'S0001'.
gs_student-name  = '王小明'.
gs_student-score = 85.

WRITE: / gs_student-id, gs_student-name, gs_student-score.
```

`結構-欄位` 是 ABAP 最高頻的寫法之一。也因此**變數命名不要含連字號**，避免與欄位存取混淆。

**新式寫法對照（7.40 之後）：`VALUE #( )` 填結構**

```
* 傳統寫法：一個欄位一行
gs_student-id    = 'S0001'.
gs_student-name  = '王小明'.
gs_student-score = 85.

* 新式寫法：一次填好整個結構
gs_student = VALUE #( id = 'S0001' name = '王小明' score = 85 ).
```

語法拆解：

- `VALUE 型別( 欄位 = 值 欄位 = 值 ... )`：建出一個「填好值的結構」，再整筆賦值給左邊。欄位之間用空白分隔，不用逗號。
- `#`：型別由左邊的目標決定，這裡就是 `gs_student` 的型別 `ty_student`。
- **沒寫到的欄位是初始值**：`VALUE #( )` 是重新建一整筆，不是只改那幾個欄位。所以它等於「先 `CLEAR gs_student`，再逐欄賦值」；如果只想改其中一個欄位、其他欄位保留原值，還是用 `gs_student-`

`score = 90.`。

練習 3 的新式寫法版本見答案程式 `ZR_TR03_STRUCTURES_NEW`（輸出跟原版逐行比對相同）。

## 5. 結構的整體操作

### 5.1 同型別結構：直接賦值

```
DATA: gs_a TYPE ty_student,
      gs_b TYPE ty_student.

gs_a-id = 'S0001'. gs_a-name = '王小明'. gs_a-score = 85.
gs_b = gs_a.                    " 整筆複製，所有欄位一次搬完
```

### 5.2 CLEAR：整筆歸零

```
CLEAR gs_a.                     " 所有欄位回到初始值
```

> 習慣：重複使用同一個結構填資料時（下一講 APPEND 前），先 CLEAR，避免上一筆殘留值混進來——這是實務上很常見的 bug 來源。

### 5.3 MOVE-CORRESPONDING：不同結構，同名欄位對搬

```
TYPES: BEGIN OF ty_student_lite,
         id   TYPE c LENGTH 5,
         name TYPE string,
       END OF ty_student_lite.

DATA: gs_full TYPE ty_student,
      gs_lite TYPE ty_student_lite.

gs_full-id = 'S0001'. gs_full-name = '王小明'. gs_full-score = 85.
MOVE-CORRESPONDING gs_full TO gs_lite.   " 只搬同名欄位 id、name
```

規則：**欄位名相同**就搬（逐欄轉換），目的端沒有的欄位丟掉，目的端多的欄位不動。欄位名拼錯就默默不搬——不會報錯，要自己核對。

**新式寫法對照（7.40 之後）：`CORRESPONDING #( )`**

```
* 傳統寫法
MOVE-CORRESPONDING gs_full TO gs_lite.

* 新式寫法
gs_lite = CORRESPONDING #( gs_full ).
```

```
* 新式寫法，保留目的端原本的值
gs_lite = CORRESPONDING #( BASE ( gs_lite ) gs_full ).
```

語法拆解：`CORRESPONDING 型別( 來源 )` 依欄位名稱對搬，結果整筆賦值給左邊；`#` 一樣是型別取自左邊的目標。

**兩者不完全相同**，差在目的端「多出來、來源沒有」的欄位（2026-10-08 實測，`ZR_TR04_NEW_CHK`，目的端多一個 `memo` 欄位、原值 `KEEP`）：

| 寫法 | `memo` 的結果 |
|---|---|
| `MOVE-CORRESPONDING gs_full TO gs_lite.` | 保留 `KEEP` |
| `gs_lite = CORRESPONDING #( gs_full ).` | **被清成空白** |
| `gs_lite = CORRESPONDING #( BASE ( gs_lite ) gs_full ).` | 保留 `KEEP` |

原因：`CORRESPONDING #( )` 是先建一個全新的結構、只填同名欄位，再整筆賦值，所以其他欄位都是初始值。加 `BASE ( gs_lite )` 表示「以 `gs_lite` 現在的內容為底」再覆蓋同名欄位，效果才跟 `MOVE-CORRESPONDING` 一樣。把舊程式的 `MOVE-CORRESPONDING` 改成新寫法時，要注意目的端是否有需要保留的欄位。

## 6. 巢狀結構（Nested Structure，先看得懂）

結構的欄位也可以是另一個結構，用兩層 `-` 存取：

```
TYPES: BEGIN OF ty_address,
         city   TYPE string,
         street TYPE string,
       END OF ty_address.
TYPES: BEGIN OF ty_person,
         name    TYPE string,
         address TYPE ty_address,      " 欄位本身是結構
       END OF ty_person.

DATA gs_person TYPE ty_person.
gs_person-address-city = '台北'.
```

實務上 SAP 標準結構常有巢狀，先能讀懂即可。

> 這裡的巢狀是「欄位裡包另一個結構」（1 對 1）。學完講義 4 / 5 的 Internal Table 之後，還會遇到「欄位裡包一整張表」（1 對多）的進階版——稱為 **Deep Structure**，見講義 5 第 9 節。

## 7. 常見錯誤與陷阱

| 症狀 | 原因 |
|---|---|
| 對 `ty_xxx` 賦值編譯錯誤 | TYPES 是型別不是變數，要先 DATA 出來 |

| 症狀 | 原因 |
| --- | --- |
| END OF 名稱打錯報錯 | BEGIN OF 與 END OF 的名稱必須相同 |
| MOVE-CORRESPONDING 後某欄位是空的 | 兩邊欄位名不同（拼錯），它只認同名欄位 |
| 這一筆混到上一筆的值 | 填欄位前忘了 CLEAR 結構 |
| 不同型別結構直接 = | 會照「位置」硬轉，結果錯亂——不同結構請用 MOVE-CORRESPONDING |

## 8. 課堂練習

完成 ex03：定義學生結構、填值輸出、練習整體賦值與 MOVE-CORRESPONDING。